

Objetivos

- Compreender a estrutura de nós e ponteiros em árvores binárias.
 - Implementar percursos (pré-ordem, em-ordem, pós-ordem, nível-a-nível).
 - Realizar inserção e remoção em Árvore Binária de Busca (ABB).
 - Analisar complexidade e comportamento recursivo.
-

Estrutura base da árvore

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

// Definição do nó
typedef struct No {
    int dado;
    struct No* esquerda;
    struct No* direita;
} No;

// Função para criar um novo nó
No* criarNo(int valor) {
    No* novo = (No*)malloc(sizeof(No));
    novo->dado = valor;
    novo->esquerda = NULL;
    novo->direita = NULL;
    return novo;
}
```

Atividade 1 – Construção e Percursos Recursivos

Código para completar. Complete as funções de percurso recursivo e a função que conta o número de nós.

```
// Percurso pré-ordem (raiz -> esquerda -> direita)
void preOrdem(No* raiz) {
    // COMPLETE AQUI
}

// Percurso em-ordem (esquerda -> raiz -> direita)
void emOrdem(No* raiz) {
    // COMPLETE AQUI
}

// Percurso pós-ordem (esquerda -> direita -> raiz)
void posOrdem(No* raiz) {
    // COMPLETE AQUI
}

// Retorna o número total de nós na árvore
int contarNos(No* raiz) {
    // COMPLETE AQUI
}

// Retorna a altura da árvore (maior distância da raiz até folha)
int altura(No* raiz) {
    // COMPLETE AQUI
}

int main() {
    // Construindo uma árvore manualmente:
    //      10
    //     /  \
    //    5   15
    //   / \   \
    //  3  7   20
    No* raiz = criarNo(10);
    raiz->esquerda = criarNo(5);
    raiz->direita = criarNo(15);
    raiz->esquerda->esquerda = criarNo(3);
    raiz->esquerda->direita = criarNo(7);
}
```

```

    raiz->direita->direita = criarNo(20);

    printf("Pre-ordem: ");
    preOrdem(raiz); // Deveria: 10 5 3 7 15 20
    printf("\n");

    printf("Em-ordem: ");
    emOrdem(raiz); // Deveria: 3 5 7 10 15 20
    printf("\n");

    printf("Pos-ordem: ");
    posOrdem(raiz); // Deveria: 3 7 5 20 15 10
    printf("\n");

    printf("Total de nos: %d\n", contarNos(raiz)); // 6
    printf("Altura: %d\n", altura(raiz)); // 3 (caminho 10-15-20)

    return 0;
}

```

Perguntas

1. Qual a principal aplicação prática do percurso em-ordem em uma ABB?
2. Por que os percursos recursivos são naturalmente adequados para árvores?
3. O que aconteceria se chamássemos `preOrdem(NULL)`? Como evitar problemas?

Atividade 2 – Árvore Binária de Busca (ABB): Inserção

Código para completar: Implemente inserção em ABB (valores menores à esquerda, maiores à direita) e a função de busca otimizada.

```

// Insere um valor na ABB mantendo a propriedade (sem repetições)
No* inserirABB(No* raiz, int valor) {
    // COMPLETE AQUI
    // Dica: se raiz == NULL, crie um novo nó e retorne

```

```

        // Caso contrário, compare valor com raiz->dado e insira na subárvore adequada
    }

    // Busca otimizada em ABB (aproveita a ordenação)
    bool buscarABB(No* raiz, int alvo) {
        // COMPLETE AQUI
        // Dica: se raiz == NULL, retorne false
        // Se alvo == raiz->dado, true
        // Se alvo < raiz->dado, busque na esquerda, senão na direita
    }

    // Função para liberar toda a memória da árvore (pós-ordem)
    void liberarArvore(No* raiz) {
        if (raiz == NULL) return;
        liberarArvore(raiz->esquerda);
        liberarArvore(raiz->direita);
        free(raiz);
    }

    int main() {
        No* raizABB = NULL;
        int valores[] = {50, 30, 70, 20, 40, 60, 80, 35, 45};
        int n = sizeof(valores) / sizeof(valores[0]);

        for (int i = 0; i < n; i++) {
            raizABB = inserirABB(raizABB, valores[i]);
        }

        printf("Em-ordem da ABB: ");
        emOrdem(raizABB); // Deve imprimir em ordem crescente
        printf("\n");

        printf("Buscar 45: %s\n", buscarABB(raizABB, 45) ? "Sim" : "Nao");
        printf("Buscar 100: %s\n", buscarABB(raizABB, 100) ? "Sim" : "Nao");

        liberarArvore(raizABB);
        return 0;
    }

```

Perguntas

1. Por que em uma ABB o percurso em-ordem resulta em valores ordenados?
 2. Qual a complexidade de tempo da inserção e busca no pior caso? E no caso médio?
 3. Insira manualmente os valores 1, 2, 3, 4, 5 (nessa ordem) em uma ABB e desenhe a árvore resultante. Isso é problemático? Por quê?
-

Atividade 3 – Remoção em ABB – Desafio

Código para completar: Implemente a remoção em ABB, tratando os três casos:

1. Nó folha (sem filhos)
2. Nó com um filho
3. Nó com dois filhos (substituir pelo menor da subárvore direita ou maior da esquerda)

```
// Encontra o nó com o menor valor (mais à esquerda)
No* menorValor(No* raiz) {
    // COMPLETE AQUI
}

// Remove um valor da ABB
No* removerABB(No* raiz, int valor) {
    if (raiz == NULL) {
        return NULL;
    }

    // Busca o nó a ser removido
    if (valor < raiz->dado) {
        raiz->esquerda = removerABB(raiz->esquerda, valor);
    }
    else if (valor > raiz->dado) {
        raiz->direita = removerABB(raiz->direita, valor);
    }
    else {
        // Nó encontrado!

        // CASO 1: sem filhos ou com um filho
        if (raiz->esquerda == NULL) {
```

```

        // COMPLETE AQUI (nó com filho direito ou folha)
    }
    else if (raiz->direita == NULL) {
        // COMPLETE AQUI (nó com filho esquerdo)
    }
    else {
        // CASO 2: dois filhos
        // Encontra o menor nó da subárvore direita
        // COMPLETE AQUI
    }
}
return raiz;
}

int main() {
    No* raiz = NULL;
    int valores[] = {50, 30, 70, 20, 40, 60, 80, 35, 45};
    for (int i = 0; i < 9; i++) raiz = inserirABB(raiz, valores[i]);

    printf("ABB original (em-ordem): ");
    emOrdem(raiz);
    printf("\n");

    // Remover folha (20)
    raiz = removerABB(raiz, 20);
    printf("Apos remover 20: ");
    emOrdem(raiz);
    printf("\n");

    // Remover nó com um filho (30 tem filho 40)
    raiz = removerABB(raiz, 30);
    printf("Apos remover 30: ");
    emOrdem(raiz);
    printf("\n");

    // Remover nó com dois filhos (50)
    raiz = removerABB(raiz, 50);
    printf("Apos remover 50: ");
    emOrdem(raiz);
    printf("\n");

    liberarArvore(raiz);
}

```

```
    return 0;  
}
```

Perguntas

1. Por que, no caso de dois filhos, escolhemos o menor da subárvore direita (ou o maior da esquerda)?
2. Implemente uma função que encontra o maior valor da subárvore esquerda como alternativa para remoção com dois filhos.
3. O que acontece se tentarmos remover um valor que não existe na árvore? A implementação proposta trata isso corretamente?
4. (Bônus) Implemente uma função que verifica se uma árvore binária é uma ABB válida.